Πολυδιάστατες Δομές Δεδομένων και Υπολογιστική Γεωμετρία

Πίνακας Περιεχομένων

Δεδομένα 3
Range Queries 4
Similarity Queries 4
Κώδικες 5
compare_utils.py 5
create_data.py 6
kd_tree.py 7
kd_tree_old.py 9
lsh.py 11

quad_tree.py 13
r_tree.py 14
run_queries.py 16
Αποτελέσματα 17

PROJECT ΠΟΛΥΔΙΑΣΤΑΤΕΣ ΔΟΜΕΣ 2022-2023

ΔΗΜΗΤΡΟΠΟΥΛΟΣ ΧΑΡΑΛΑΜΠΟΣ 1080435
ΚΑΜΑΤΣΟΣ ΠΡΟΚΟΠΗΣ 1072586
ΣΤΡΑΒΟΔΗΜΟΣ ΠΕΡΙΚΛΗΣ 1080436
ΦΙΛΙΠΠΟΠΟΥΛΟΣ ΚΩΝΣΤΑΝΤΙΝΟΣ 1080437
ΧΡΙΣΤΟΔΟΥΛΙΔΗΣ ΜΙΧΑΛΗΣ 1063020

Σκοπός της εργασίας ήταν να αναπτύξουμε πολυδιάστατα ευρετήρια για την εκτέλεση range queries πάνω σε δεδομένα, και έπειτα εκτέλεση ερωτημάτων ομοιότητας.

Δεδομένα

Τα δεδομένα με τα οποία καλούμαστε να δουλέψουμε είναι μεγάλοι επιστήμονες των υπολογιστών, μετα βραβεία που έχουν αποσπάσει καθώς και το γνωστικό τους αντικείμενο.

Καθώς η πληροφορία δεν ήταν μαζεμένη σε κάποιο dataset συγκεντρωτικά, και καθώς η συγκέντρωση των ορθών δεδομένων θα ήταν αρκετά χρονοβόρα, κατασκευάσαμε ψεύτικα δεδομένα για τον αριθμό βραβείων και το γνωστικό αντικείμενο του κάθε επιστήμονα.

Το αρχείο create_data.py κατασκευάζει το dataset. Αρχικά, παίρνει από τη Wikipedia τα ονόματα των επιστημώνων, καθώς και τα πεδία της επιστήμης των Υπολογιστών από τις ανάλογες σελίδεςλίστες, με χρήση της βιβλιοθήκης BeautifulSoup με τεχνικές web scraping.

Για κάθε επιστήμονα, καυασκευάζουμε ένα τυχαίο αριθμό βραβείων με μια κανονική κατανομή, καθώς και ένα γνωστικό αντικείμενο, σαν ένα τυχαίο συνδυασμό κάποιον από τα πεδία της Επιστήμης των υπολογιστών.

Χρησιμοποιούμε τη βιβλιοθήκη pandas για να αποθηκεύσουμε το dataset με χρήση ενός DataFrame. Με τη βοήθειά της αποθηκεύουμε το dataset σε αρχείο .tsv, δηλαδή στηλοθετημένο αρχείο κειμένου, χωρισμένου με tabs.
Εδώ βλέπουμε κάποια από τα δεδομένα μας.

scientists.tsv - Notepad

File Edit Format View Help

full_names	Surname	#Awards	Education
Atta ur Rehman Khan	Atta	7	human-computer interaction, coding theory, computer architecture, information visualization, machine learning, computer security, numerical analysis,
Wil van der Aalst	Aalst	9	information visualization, data structures, relational databases, networking, computational science, bioinformatics
Scott Aaronson	Aaronson	6	computational chemistry, computational science, soft computing, image processing, reverse engineering, human-computer interaction
Rediet Abebe	Abebe	6	formal semantics, reverse engineering, distributed computing, symbolic computation, programming language theory
Hal Abelson	Abelson	13	automated reasoning, evolutionary computing, human-computer interaction, mathematical logic, machine learning, computer graphics
Serge Abiteboul	Abiteboul	12	game theory, data mining, computer graphics, quantum computing, programming language theory, algorithm design, distributed computing
Samson Abramsky	Abramsky	8	coding theory, formal methods, computer vision, computability theory, symbolic computation
Leonard Adleman	Adleman	8	cryptography, algorithms, computational neuroscience, distributed computing, computational complexity theory, software engineering
Manindra Agrawal	Agrawal	10	reverse engineering, parallel computing, natural language processing, formal semantics, computational physics, computer programming
Luis von Ahn	Ahn	11	quantum computing, discrete mathematics, natural language processing, data structures, programming language pragmatics, evolutionary computing, computational
Alfred Aho	Aho	11	networking, symbolic computation, relational databases, formal semantics, artificial intelligence, discrete mathematics
Frances E. Allen	Allen	4	computer architecture, coding theory, automated reasoning, natural language processing, image processing, type theory
Gene Amdahl	Amdahl	8	computer vision, parallel computing, operating systems, coding theory, graph theory, reverse engineering, computer programming
David P. Anderson	Anderson	8	number theory, parallel computing, soft computing, coding theory, computer programming, information visualization, evolutionary computing, so
Lisa Anthony	Anthony	12	compiler theory, computational chemistry, graph theory, coding theory, bioinformatics
Andrew Appel	Appel	10	operating systems, computability theory, computational chemistry, distributed computing, computational science, machine learning, information visualization,
Cecilia R. Aragon	Aragon	1	computational science, automated reasoning, number theory, programming language pragmatics, computer vision
Bruce Arden	Arden	6	natural language processing, evolutionary computing, computer vision, computability theory, cryptography, automated reasoning, type theory
Sanjeev Arora	Arora	13	formal semantics, algorithm design, coding theory, automated reasoning, formal methods
"Winifred ""Tim"" Alice	Asprey" Asprey	6	soft computing, computational physics, algorithms, networking, programming language theory, concurrency (computer science), mathematical logi
John Vincent Atanasoff	Atanasoff	6	soft computing, robotics, quantum computing, game theory, information visualization, computer graphics, computability theory, concurrency (co
Shakuntala Atre	Atre	9	soft computing, numerical analysis, software engineering, formal semantics, robotics, distributed computing
Charles Babbage	Babbage	17	operating systems, data structures, mathematical logic, computational neuroscience, data mining, machine learning, bioinformatics, computational chemistry
Charles Bachman	Bachman	4	reverse engineering, artificial intelligence, formal semantics, natural language processing, computer graphics, numerical analysis
Roland Carl Backhouse	Backhouse	6	artificial intelligence, type theory, programming language theory, algorithm design, formal methods, graph theory, networking
John Backus	Backus	8	discrete mathematics, information visualization, formal semantics, type theory, numerical analysis, algorithms, bioinformatics, mathematical logic
David F. Bacon	Bacon	8	information visualization, computer programming, bioinformatics, discrete mathematics, computer security, structured storage
David A. Bader	Bader	9	concurrency (computer science), computer architecture, structured storage, computational chemistry, networking, computational complexity theory, programming
Victor Bahl	Bahl	8	graph theory, networking, structured storage, game theory, natural language processing, image processing, artificial intelligence, coding theory
Anthony James Barr	Barr	12	game theory, data structures, computational physics, machine learning, concurrency (computer science)
Jean Bartik	Bartik	5	soft computing, computer security, image processing, evolutionary computing, computer architecture, human-computer interaction, operating systems, parallel c
Andrew Barto	Barto	5	computational chemistry, computer architecture, information visualization, computational complexity theory, discrete mathematics
Friedrich L. Bauer	Bauer	0	machine learning, computer vision, number theory, automated reasoning, computer architecture, natural language processing, computer graphics, relatio
Rudolf Bayer	Bayer	2	coding theory, automated reasoning, data structures, programming language theory, discrete mathematics, computer graphics
Gordon Bell	Bell	5	operating systems, discrete mathematics, robotics, algorithms, programming language pragmatics, automated reasoning, symbolic computation, evolutionary compu
Steven M. Bellovin	Bellovin	9	numerical analysis, type theory, image processing, computer vision, reverse engineering, cryptography, discrete mathematics
Cecilia Berdichevsky	Berdichevsky	4	algorithms, formal semantics, evolutionary computing, computational science, programming language theory
Tim Berners-Lee	Berners-Lee	8	formal semantics, coding theory, computability theory, quantum computing, automated reasoning, symbolic computation, type theory, operating systems
Daniel J. Bernstein	Bernstein	4	data mining, distributed computing, structured storage, game theory, information visualization, formal methods
Peter Bernus	Bernus	8	parallel computing, image processing, software engineering, type theory, formal methods
Abhay Bhushan	Bhushan	0	operating systems, compiler theory, structured storage, discrete mathematics, parallel computing

Range Queries

Ένα range query αναζητά δεδομένα που βρίσκονται μεταξύ κάποιων ορίων, σε έναν ή και περισσότερους άξονες-διαστάσεις. Στη περίπτωση της εργασίας και των δεδομένων μας, τα range queries γίνονται για τα πεδία του ονόματος και του αριθμού βραβείων των επιστημώνων.

Χωρίς ευρετήρια, ένα range query σε k διαστάσεις ενός dataset n εγγραφών, θα απαιτούσε O(nk) χρόνο για την εύρεση των εγγραφών, καθώς τα δεδομένα θα έπρεπε να ελεγχουν διαδοχικά για το αν ανήκουν μέσα στο range που έχει καθοριστεί για την εκάστοτε διάσταση.

Η δημιουργία ευρετηρίων μειώνει αυτό το χρόνο, αποθηκεύοντας τα δεδομένα σε δομές ανάλογα με τη θέση τους στο χώρο των διαστάσεων του ευρετηρίου.

Στα πλαίσια της εργασίας κατασκευάσαμε ευρετήρια με χρήση **kd-trees, quad-trees, range-trees, rtrees.**

Similarity Queries

Ένα ερώτημα ομοιότητας προσπαθεί να δώσει ένα μετρικό ομοιότητας σε δύο εγγραφές, και τα επιστρέψει αυτές που περνάνε ένα κατώφλι. Για να προσδιορίσουμε αυτό το μετρικό, μετατρέπουμε το κείμενο που πρέπει να συγκρίνουμε σε ένα σύνολο από shingles, μικρά δηλαδή (μήκος 2 χαρακτήρων) επικαληπτώμενα τμήματα. Ορίζουμε την **ομοιότητα Jaccard** δύο κειμένων ως το μέγεθος της τομής αυτών των συνόλων προς το μέγεθος της ένωσης αυτών.

Εύκολα διαπιστώνουμε πως η εκτέλεση των συγκεκριμένων πράξεων πάνω σε όλα τα ζευγάρια που θα προκύψουν από κάποιο range query πάνω στα δεδομένα μας, είναι πολυ χρονοβόρα.

Υλοποιούμε, λοιπόν, τη τεχνική του **Local Sensitive Hashing (LSH)**. Αυτή η τεχνική δίνει μια ταυτότητα (λίστα από αριθμούς) σε κάθε μία εγγραφή κειμένου, βασιζόμενη στα shingles που προκύπτουν από αυτό, με την ιδιότητα πως παραπλήσια κείμενα-σύνολο shingles δίνουν παραπλήσιες signatures.

Αυτά τα signatures έπειτα χωρίζουμε σε μικρότερα τμήματα, βάσει των οποίων βάζουμε σε buckets τις εγγραφές. Αν δύο εγγραφές “πέσουν” στο ίδιο bucket, τις χαρακτηρίσουμε ως υποψήφιες και εκτελούμε σε αυτές **Jaccard,** για να διαπιστώσουμε αν το ζευγάρι είναι τελικά όμοιο.

Καθώς η χρήση των συνόλων δεν είναι ουτοπική, αλλά κάθε αναζήτηση και εκτέλεση ένωσης η τομής απαιτεί χρόνο τάξης O(logn) και τουλάχιστον O(n+m) αντίστοιχα, αποθηκεύουμε τα σύνολα σε μορφή one-hots. Τα one-hots δημιουργούν μια λίστα με όλα τα μοναδικά shingles σε όλες τις εγγραφές, το οποίο λέμε vocabulary. Έτσι, κάθε άλλο σύνολο μετατρέπεται σε ένα bit array μεγέθους όσο το vocabulary, στα οποία 1 σημαίνει πως το shingle στην αντίστοιχη θέση του vocabulary ανήκει σε αυτό το σύνολο. Έτσι, μετατρέπουμε τις πράξεις τομής και ένωσης σε απλές OR και AND.

Κώδικες

Πιο κάτω παραθέτουμε τους κώδικες που έχουμε υλοποιήσει

compare_utils.py

```
RANGE_NINF = "ninf"
RANGE_INF  = "inf"

EQUAL = 0
LESS_THAN = -1
MORE_THAN = 1

RANGE_SUBRANGE = 1
RANGE_INTERCEPT = 2
RANGE_NORANGE = 3

def compare_all(a, b):
    if a == b:
        return EQUAL
    elif a == RANGE_NINF:
        return LESS_THAN
    elif a == RANGE_INF:
        return MORE_THAN
    elif b == RANGE_NINF:
        return MORE_THAN
    elif b == RANGE_INF:
        return LESS_THAN
    elif a < b:
        return LESS_THAN
    else:
        return MORE_THAN


class RangeData:

    def __init__(self, l, h):
        self.low = l
        self.high = h


def compare_range(ra, rb):
    state = RANGE_SUBRANGE
    for dima, dimb in zip(ra, rb):
        down = compare_all(dima.low, dimb.low)
        up   = compare_all(dima.high, dimb.high)

        if compare_all(dima.high, dimb.low) == LESS_THAN or
compare_all(dima.low, dimb.high) == MORE_THAN:
            return RANGE_NORANGE
        elif compare_all(dima.low, dimb.low) == LESS_THAN or
compare_all(dima.high, dimb.high) == MORE_THAN:
            state = RANGE_INTERCEPT

    return state
```

create_data.py

```
import pandas as pd
import numpy as np
from bs4 import BeautifulSoup
import requests
import random
import os.path


NAMES_URL = "https://en.wikipedia.org/wiki/List_of_computer_scientists"
FIELDS_URL = "https://en.wikipedia.org/wiki/Outline_of_computer_science"


html_doc = requests.get(NAMES_URL)
soup = BeautifulSoup(html_doc.content, "html.parser")
data_div = soup.find("div", "mw-parser-output")

initial_letter = ""
cs_names = []
first_names = []

for el in data_div.find_all(["li", "h2"]):
    if el.name == "h2":
        initial_letter = el.span.string
    else:
        name = el.a.string
        segs = name.split()
        if segs[-1].title().startswith(initial_letter):
            fname = segs[-1]
        else:
            fname = segs[0]

        cs_names.append(name)
        first_names.append(fname)

f = cs_names.index("Atta ur Rehman Khan")
t = cs_names.index("Konrad Zuse")

cs_names = cs_names[f:t+1]
first_names = first_names[f:t+1]

html_doc = requests.get(FIELDS_URL)
soup = BeautifulSoup(html_doc.content, "html.parser")
data_div = soup.find("div", "mw-parser-output")

cs_fields = [li.a.string for li in data_div.find_all("li")]
cs_fields = list(filter(lambda x: not x.startswith("List"), filter(None,
cs_fields)))
cs_fields = cs_fields[cs_fields.index("Coding theory"):
                      cs_fields.index("Quantum computing")+1]

# warning: fake data!

SEED = 12345
np.random.seed(SEED)
random.seed(SEED)
# create random number of awards
```

```
awards = list(map(lambda x: max(int(x), 0), np.random.normal(8, 3,
size=len(cs_names))))

# create random education for each scientist
education = [", ".join(random.sample(cs_fields, k=random.randint(5, 8))).lower()
             for _ in range(len(cs_names))]



df = pd.DataFrame({
    "full_names": cs_names,
    "Surname": first_names,
    "#Awards": awards,
    "Education": education
})

# write output to file
df.to_csv(os.path.join("data", "scientists.tsv"), index=False, sep='\t')

print(len(cs_fields))
```

kd_tree.py

```
import numpy as np
import pandas as pd
import compare_utils as cu


class KDNode:

    node_count = 0
    indexed_nodes = 0
    def __init__(self, tree, items, depth=0, forward_count=0):
        KDNode.node_count += 1
        self.tree = tree
        self.axis = depth % len(self.tree.axis_list)
        self.axis_name = self.tree.axis_list[self.axis]

        self.right_child = None
        self.left_child = None

        # find median based on axis
        items.sort(key=lambda x:self.tree.dataset[self.axis_name][x])
        median_index = len(items) // 2

        self.median_value =
self.tree.dataset[self.axis_name][items[median_index]]
        self.median = items[median_index]
        KDNode.indexed_nodes += 1

        # create new index lists
        left_items = items[:median_index]
        right_items = items[median_index+1:]

        if len(left_items) > 0:
            self.left_child = KDNode(self.tree, left_items, depth+1)
```

```
        if len(right_items) > 0:
            self.right_child = KDNode(self.tree, right_items, depth+1)



    def report_subtree(self):
        ret = [self.median]
        if self.left_child is not None:
            ret.extend(self.left_child.report_subtree())

        if self.right_child is not None:
            ret.extend(self.right_child.report_subtree())

        return ret

    def range_search(self, range_data, current_range):
        left_range = [cu.RangeData(r.low, r.high) for r in current_range]
        left_range[self.axis].high = self.median_value
        right_range = [cu.RangeData(r.low, r.high) for r in current_range]
        right_range[self.axis].low = self.median_value

        el_range = [cu.RangeData(self.tree.dataset[ax][self.median],
self.tree.dataset[ax][self.median]) for ax in self.tree.axis_list]
        results = []
        if cu.compare_range(el_range, range_data) == cu.RANGE_SUBRANGE:
            results.append(self.median)

        if self.left_child is not None:
            lcr = cu.compare_range(left_range, range_data)
            if lcr == cu.RANGE_SUBRANGE:
                results.extend(self.left_child.report_subtree())
            elif lcr == cu.RANGE_INTERCEPT:
                results.extend(self.left_child.range_search(range_data,
left_range))

        if self.right_child is not None:
            rcr = cu.compare_range(right_range, range_data)
            if rcr == cu.RANGE_SUBRANGE:
                results.extend(self.right_child.report_subtree())
            elif rcr == cu.RANGE_INTERCEPT:
                results.extend(self.right_child.range_search(range_data,
right_range))


        return results


class KDTree:

    def __init__(self, dataset, axis_list):
        # pre-sort indexes for each col
        self.dataset = dataset
        self.axis_list = axis_list
        indexes = list(range(len(self.dataset)))

        # build kd tree
        self.root = KDNode(self, indexes)
```

```
    def range_search(self, range_data):
        c_range = [cu.RangeData(cu.RANGE_NINF, cu.RANGE_INF) for _ in
range(len(self.axis_list))]

        return self.root.range_search(range_data, c_range)
```

kd_tree_old.py

```
import numpy as np
import pandas as pd
import compare_utils as cu


class KDNode:

    node_count = 0
    indexed_nodes = 0
    def __init__(self, tree, indexes, depth=0, forward_count=0):
        KDNode.node_count += 1
        self.tree = tree
        self.axis = depth % len(self.tree.axis_list)
        self.axis_name = self.tree.axis_list[self.axis]
        # next_axis = (self.axis+1) % len(self.tree.axis_list)

        self.right_child = None
        self.left_child = None

        if forward_count == len(tree.axis_list):
            self.terminal_node = True
            self.terminal_values = indexes[self.axis]
            KDNode.indexed_nodes += len(self.terminal_values)
            return


        self.median_index = len(indexes[self.axis]) // 2
        if len(indexes[self.axis]) % 2 == 0:
            self.median_index -= 1
        self.median_value =
tree.dataset[self.axis_name][indexes[self.axis][self.median_index]]
        self.terminal_node = len(indexes[self.axis]) == 1

        if self.terminal_node:
            self.terminal_values = indexes[self.axis]
            KDNode.indexed_nodes += len(self.terminal_values)
            return

        # print(f"depth={depth} axis{self.axis_name}
median_value={self.median_value} tnode {self.terminal_node}")

        # create new index lists
        left_indexes = []
        right_indexes = []
        for ax_idx, ax_name in enumerate(tree.axis_list):
            l = []
            r = []
            for i_pos in indexes[ax_idx]:
```

```
                if tree.dataset[self.axis_name][i_pos] <= self.median_value:
                    l.append(i_pos)
                else:
                    r.append(i_pos)

            left_indexes.append(l)
            right_indexes.append(r)

            # print(f"Split {ax_name}: {len(l)} | {len(r)}")

        if len(left_indexes[self.axis]) == 0:
            # print("forwarding")
            self.right_child = KDNode(self.tree, indexes, depth+1,
forward_count+1)
            self.left_child = None
        elif len(right_indexes[self.axis]) == 0:
            self.left_child = KDNode(self.tree, indexes, depth+1,
forward_count+1)
            self.right_child = None
        else:
            self.left_child  = KDNode(self.tree, left_indexes, depth+1)
            self.right_child = KDNode(self.tree, right_indexes, depth+1)


    def report_subtree(self):
        if self.terminal_node:
            return self.terminal_values

        ret = []
        if self.left_child is not None:
            ret.extend(self.left_child.report_subtree())

        if self.right_child is not None:
            ret.extend(self.right_child.report_subtree())

        return ret

    def range_search(self, range_data, current_range):
        left_range = [cu.RangeData(r.low, r.high) for r in current_range]
        left_range[self.axis].high = self.median_value
        right_range = [cu.RangeData(r.low, r.high) for r in current_range]
        right_range[self.axis].low = self.median_value

        results = []
        el_range = [cu.RangeData(self.tree.dataset[ax][self.median_index],
self.tree.dataset[ax][self.median_index]) for ax in self.tree.axis_list]
        if self.terminal_node and cu.compare_range(el_range, range_data) ==
cu.RANGE_SUBRANGE:
            return self.terminal_values

        if self.left_child is not None:
            lcr = cu.compare_range(left_range, range_data)
            if lcr == cu.RANGE_SUBRANGE:
                results.extend(self.left_child.report_subtree())
            elif lcr == cu.RANGE_INTERCEPT:
                results.extend(self.left_child.range_search(range_data,
left_range))
```

```
        if self.right_child is not None:
            rcr = cu.compare_range(right_range, range_data)
            if rcr == cu.RANGE_SUBRANGE:
                results.extend(self.right_child.report_subtree())
            elif rcr == cu.RANGE_INTERCEPT:
                results.extend(self.right_child.range_search(range_data,
right_range))


        return results


class KDTree:

    def __init__(self, dataset, axis_list):
        # pre-sort indexes for each col
        self.dataset = dataset
        self.axis_list = axis_list
        indexes = [self.dataset[axis].argsort() for axis in self.axis_list]

        # build kd tree
        self.root = KDNode(self, indexes)

    def range_search(self, range_data):
        c_range = [cu.RangeData(cu.RANGE_NINF, cu.RANGE_INF) for _ in
range(len(self.axis_list))]

        return self.root.range_search(range_data, c_range)
```

lsh.py

```
import pandas as pd
import numpy as np
from functools import partial
from itertools import combinations
import os

def jaccard(a: set, b: set):
    return len(a.intersection(b)) / len(a.union(b))

def jaccard_onehots(a, b):
    return sum(map(all, zip(a, b))) / sum(map(any, zip(a, b)))

dataset_path = os.path.join("data", "scientists.tsv")

# load dataset
df = pd.read_csv(dataset_path, sep='\t')

# create LSH vocabulary
# data = df["Education"].str.split(", ")

# create shingles
def shingle(text, k=2):
    shingle_set = []
    for i in range(len(text) - k+1):
        shingle_set.append(text[i:i+k])
    return set(shingle_set)
```

```python
def create_onehot(shingles, vocab_index):
    onehot = np.zeros(len(vocab_index), dtype=np.uint8)
    for shingle in shingles:
        onehot[vocab_index[shingle]] = 1

    return onehot


def create_hash_line(length):
    hash_line = np.arange(length)
    np.random.shuffle(hash_line)
    return hash_line


def create_min_hash(min_hashes, length):
    return [create_hash_line(length) for _ in range(min_hashes)]


def compute_signature(vector, min_hash):
    sig = np.zeros(len(min_hash), dtype=int)

    for sig_idx, hash_line in enumerate(min_hash):
        for idx, vec_idx in enumerate(hash_line):
            if vector[vec_idx] == 1:
                sig[sig_idx] = idx
                break

    return sig

def split_signature_to_buckets(sig, buckets):
    bucket_size, rem = divmod(len(sig), buckets)
    if rem != 0:
       buckets += 1

    sig_buckets = []
    for bi, i in enumerate(range(0, len(sig), bucket_size)):
        end = i+bucket_size
        if bi == buckets-1:
            end = len(sig)
        sig_buckets.append(sig[i:end])

    return sig_buckets

# precalculate shingles
# df["education_shingles"] = df["Education"].apply(shingle)
#
# # create vocabulary
# vocabulary = list(set().union(*df["education_shingles"]))
# vocab_index = {shingle: idx for idx, shingle in enumerate(vocabulary)}
#
# df["education_onehot"] = df["education_shingles"].apply(partial(create_onehot,
vocab_index=vocab_index))
#
# MIN_HASHES = 20
# BUCKETS = 10
#
# min_hash = create_min_hash(MIN_HASHES, len(vocabulary))
```

```
# df["lsh_signature"] = df["education_onehot"].apply(partial(compute_signature,
min_hash=min_hash))

# print(df.iloc[0])
# print(split_signature_to_buckets(df.iloc[0]["lsh_signature"], BUCKETS),
sep='\n')
# print(compute_signature(test_vec, minhash))
# print(df.head())

def compute_candiadate_index_pairs(min_hashes, n_buckets):
    buckets = [dict() for _ in range(n_buckets)]

    for mhi, min_hash in enumerate(min_hashes):
        min_hash_spiltted = split_signature_to_buckets(min_hash, n_buckets)
        for si, segment in enumerate(min_hash_spiltted):
            key = '-'.join(map(str, segment))
            if key not in buckets[si]:
                buckets[si][key] = list()

            buckets[si][key].append(mhi)

    candidate_pairs = set()
    for bucket in buckets:
        for sames in bucket.values():
            for pair in combinations(sames, 2):
                candidate_pairs.add(pair)


    return list(candidate_pairs)
```

quad_tree.py

```
class QuadTreeNode:

    def __init__(self, tree, item_index, range_x, range_y):
        self.tree = tree
        self.index = index
        self.range_x = range_x
        self.range_y = range_y`

        self.points = []
        self.children = []

    def insert(self, item_index):
        if len(self.children) > 0:
            index = self.get_index(item_index)
            if index != -1:
                self.children[index].insert(item_index)
                return
        self.points.append((x, y))
        if len(self.points) > 4:
            self.subdivide()

    def get_index(self, item_index):
        vertical_midpoint = self.x + self.width / 2
```

```
        horizontal_midpoint = self.y + self.height / 2
        top_quadrant = (y < horizontal_midpoint and x < vertical_midpoint)
        bottom_quadrant = (y >= horizontal_midpoint and x < vertical_midpoint)
        if top_quadrant:
            return 1
        elif bottom_quadrant:
            return 2
        else:
            horizontal_midpoint = self.y + self.height / 2
            if y < horizontal_midpoint:
                return 0
            else:
                return 3

    def subdivide(self):
        quarter_width = self.width / 4
        quarter_height = self.height / 4
        x = self.x
        y = self.y
        self.children.append(Node(x, y, quarter_width, quarter_height))
        self.children.append(Node(x + quarter_width, y, quarter_width,
quarter_height))
        self.children.append(Node(x, y + quarter_height, quarter_width,
quarter_height))
        self.children.append(Node(x + quarter_width, y + quarter_height,
quarter_width, quarter_height))
        for point in self.points:
            index = self.get_index(point)
            self.children[index].insert(point)
        self.points = []

    def query_range(self, x_range, y_range):
        x_min, x_max = map(float, x_range.split(","))
        y_min, y_max = map(float, y_range.split(","))
        results = []
        if x_min <= self.x + self.width and x_max >= self.x and y_min <= self.y
+ self.height and y_max >= self.y:
            for point in self.points:
                if x_min <= point[0] <= x_max and y_min <= point[1] <= y_max:
                    results.append(point)
            for child in self.children:
                results += child.query_range(x_range, y_range)
        return results


class QuadTree:

    def __init__(self, dataset, axis_list):
        self.dataset = dataset
        self.axis_list = axis_list
```

r_tree.py

```
import hashlib
import compare_utils as cu

class RTreeNode:
```

```python
    def __init__(self, is_leaf=False, children=None, MBR=None):
        self.is_leaf = is_leaf
        self.children = children if children else []
        self.MBR = MBR

class RTree:
    def __init__(self, items):
        self.max_entries = 4
        self.root = RTreeNode(is_leaf=True)
        for i, row in items.iterrows():
            self.insert(i, [cu.RangeData(row["Surname"], row["Surname"]+"a"),
                            cu.RangeData(row["#Awards"], row["#Awards"]+0.1)])

    def insert(self, point, MBR):
        node = self._choose_leaf(self.root, MBR)
        node.children.append((point, MBR))
        self._split_node(node)

    def _choose_leaf(self, node, MBR):
        if node.is_leaf:
            return node
        min_enlargement = float("inf")
        best_child = None
        for child in node.children:
            child_MBR = child[1]
            enlargement = self._calculate_enlargement(MBR, child_MBR)
            if enlargement < min_enlargement:
                min_enlargement = enlargement
                best_child = child[0]
        return self._choose_leaf(best_child, MBR)

    def _split_node(self, node):
        if len(node.children) <= self.max_entries:
            return
        seeds = self._get_seeds(node)
        groups = [[], []]
        for child in node.children:
            if child == seeds[0] or child == seeds[1]:
                continue
            min_enlargement = float("inf")
            best_group = None
            for group in groups:
                enlargement = self._calculate_enlargement(child[1], node.MBR)
                if enlargement < min_enlargement:
                    min_enlargement = enlargement
                    best_group = group
            best_group.append(child)
        node.children = [RTreeNode(children=group) for group in groups]

    def _get_seeds(self, node):
        max_distance = 0
        seeds = [None, None]
        for i in range(len(node.children)):
            for j in range(i + 1, len(node.children)):
                child1 = node.children[i]
                child2 = node.children[j]
                distance = self._calculate_distance(child1.MBR, child2.MBR)
                if distance > max_distance:
```

```
                    max_distance = distance
                    seeds = [child1, child2]
        return seeds

    def _calculate_distance(self, MBR1, MBR2):
        return abs(ord(MBR1[0].low[0]) - ord(MBR2[0].low[0])) + abs(MBR1[1].low
- MBR2[1].low)

    def _calculate_enlargement(self, MBR1, MBR2):
        if MBR1 is None or MBR2 is None:
            return 0

        return abs(ord(MBR1[0].low[0]) - ord(MBR2[0].low[0])) + abs(MBR1[1].low
- MBR2[1].low)

    def range_search(self, region):
        return self._range_query(self.root, region)

    def _range_query(self, node, region):
        if node.is_leaf:
            return [child for child in node.children if
cu.compare_range(child[1], region) == cu.RANGE_INTERCEPT]
        results = []
        for child in node.children:
            if cu.compare_range(child[1], region) == cu.RANGE_INTERCEPT:
                results.extend(self._range_query(child[0], region))
        return results
```

run_queries.py

```
import pandas as pd
import os
import compare_utils as cu
from functools import partial
from lsh import *

from kd_tree import KDTree

MIN_HASHES = 20
BUCKETS = 10
# read the data
scientists = pd.read_csv(os.path.join("data", "scientists.tsv"), sep='\t')
index_columns = ("Surname", "#Awards")

# precalculate shingles
scientists["education_shingles"] = scientists["Education"].apply(shingle)

# create vocabulary
vocabulary = list(set().union(*scientists["education_shingles"]))
vocab_index = {shingle: idx for idx, shingle in enumerate(vocabulary)}

scientists["education_onehot"] =
scientists["education_shingles"].apply(partial(create_onehot,
vocab_index=vocab_index))
min_hash = create_min_hash(MIN_HASHES, len(vocabulary))
# ["lsh_signature"] = df["education_onehot"].apply(partial(compute_signature,
min_hash=min_hash))
```

```
print("all records", len(scientists))
kd = KDTree(scientists, index_columns)
#rt = RangeTree(scientists, index_columns[::-1]) :RangeTree

def print_results(results):
    print(scientists[["Surname", "#Awards", "Education"]].iloc[results])


# print(signatures[0])

def query(index_tree, query_range, similarity_threshold):
    res = index_tree.range_search(query_range)
    signatures =
scientists["education_onehot"].iloc[res].apply(partial(compute_signature,
min_hash=min_hash))
    similar = compute_candiadate_index_pairs(signatures, BUCKETS)

    results = []
    for sa, sb in similar:
        ga = res[sa]
        gb = res[sb]

        similarity = jaccard_onehots(*scientists["education_onehot"].iloc[[ga,
gb]])
        if similarity >= similarity_threshold:
            results.append((ga, gb, similarity))

    return results


def print_query_results(results):
    for ga, gb, similarity in results:
        print(ga, scientists["Surname"][ga], scientists["Education"][ga])
        print(gb, scientists["Surname"][gb], scientists["Education"][gb])
        print(f"Similarity: {similarity*100:.2f}%")


test_range = [cu.RangeData("A", "Brooks"), cu.RangeData(0, 5)]
qr = query(kd, test_range, 0.60)
print_query_results(qr)
```

Αποτελέσματα

Αν τρέξουμε αυτό το παράδειγμα τα αποτελέσματα που θα εμφανιστούν είναι τα πιο κάτω

```
test_range = [cu.RangeData("A", "Brooks"), cu.RangeData(0, 5)]
qr = query(kd, test_range, 0.60)
print_query_results(qr)
```

IDLE Shell 3.11.2

File Edit Shell Debug Options Window Help

```
Python 3.11.2 (tags/v3.11.2:878ead1, Feb  7 2023, 16:38:35) [MSC v.1934 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: D:\Documents\Ceid\5ο έτος\9ο εξάμηνο\Πολυδιάστατες Δομές Δεδομένων κα
ι Υπολογιστική Γεωμετρία\Project_PDDYG_2023\src\run_queries.py
all records 672
32 Bauer machine learning, computer vision, number theory, automated reasoning,
computer architecture, natural language processing, computer graphics, relationa
l databases
11 Allen computer architecture, coding theory, automated reasoning, natural lang
uage processing, image processing, type theory
Similarity: 64.36%
23 Bachman reverse engineering, artificial intelligence, formal semantics, natur
al language processing, computer graphics, numerical analysis
44 Blei natural language processing, computer graphics, reverse engineering, com
puter vision, parallel computing, symbolic computation
Similarity: 61.39%
33 Bayer coding theory, automated reasoning, data structures, programming langua
ge theory, discrete mathematics, computer graphics
16 Aragon computational science, automated reasoning, number theory, programming
 language pragmatics, computer vision
Similarity: 60.00%
70 Brooks programming language pragmatics, structured storage, algorithm design,
 coding theory, data mining, information visualization
69 Brinkkemper programming language pragmatics, artificial intelligence, computa
tional physics, automata theory, information visualization, algorithm design, nu
merical analysis, computer architecture
Similarity: 60.66%
32 Bauer machine learning, computer vision, number theory, automated reasoning,
computer architecture, natural language processing, computer graphics, relationa
l databases
16 Aragon computational science, automated reasoning, number theory, programming
 language pragmatics, computer vision
Similarity: 63.81%
>>>
```

Εδώ βλέπουμε άλλο παράδειγμα με περισσότερα βραβεία

```
test_range = [cu.RangeData("A", "Azz"), cu.RangeData(0, 99)]
qr = query(kd, test_range, 0.60)
print_query_results(qr)
```

```
all records 672
11 Allen computer architecture, coding theory, automated reasoning, natural lang
uage processing, image processing, type theory
17 Arden natural language processing, evolutionary computing, computer vision, c
omputability theory, cryptography, automated reasoning, type theory
Similarity: 62.89%
```